

Set 1:

```
In [30]: import numpy as np
```

Create a 3×3 array of all True Boolean values

```
In [31]: b_arr = np.ones((3,3), dtype = bool)
b_arr
```

```
Out[31]: array([[ True,  True,  True],
               [ True,  True,  True],
               [ True,  True,  True]])
```

Extract odd numbers from an array

```
In [32]: arr = np.arange(12).reshape(3,4)
mask = arr%2!=0
mask
```

```
Out[32]: array([[False,  True, False,  True],
               [False,  True, False,  True],
               [False,  True, False,  True]])
```

```
In [33]: arr[mask]
```

```
Out[33]: array([ 1,  3,  5,  7,  9, 11])
```

Replace all odd numbers with -1

```
In [34]: arr1 = arr[mask]
arr1
```

```
Out[34]: array([ 1,  3,  5,  7,  9, 11])
```

```
In [35]: arr
```

```
Out[35]: array([[ 0,  1,  2,  3],
               [ 4,  5,  6,  7],
               [ 8,  9, 10, 11]])
```

Reverse columns of a 2D array

```
In [36]: arr[:,::-1]
```

```
Out[36]: array([[ 3,  2,  1,  0],
               [ 7,  6,  5,  4],
               [11, 10,  9,  8]])
```

Get indices of elements greater than 5

```
In [37]: # print('arr : ',arr)
arr = np.array(arr)
# print('arr : ',arr)
result = np.where(arr>5, 1,0)
print('result : ',result)
# print('arr : ',arr[result])
```

```
result : [[0 0 0 0]
          [0 0 1 1]
          [1 1 1 1]]
```

```
In [38]: arr2 = np.arange(1,10).reshape(3,3)
arr2
# arr2 = np.argwhere(arr2>5)
```

```
Out[38]: array([[1, 2, 3],
               [4, 5, 6],
               [7, 8, 9]])
```

```
In [39]: arr2 = np.arange(1,10).reshape(3,3)
arr2_moreThan5 = np.where(arr2>5)
arr2_moreThan5
```

```
Out[39]: (array([1, 2, 2, 2], dtype=int64), array([2, 0, 1, 2], dtype=int64))
```

Access last row - last column

```
In [40]: arr2[-1,-1]
```

```
Out[40]: 9
```

Set 2:

Normalize a NumPy array [to normalize means scaling the values of an array into a specific range, often [0, 1]]

?

Create a 5×5 array with 1 on border and 0 inside

```
In [41]: arr2 = np.ones((5,5))
```

```
In [42]: arr2
```

```
Out[42]: array([[1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1.]])
```

```
In [43]: arr2[1:-1,1:-1] = 0
```

```
In [44]: print(arr2)
```

```
[[1. 1. 1. 1. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 1. 1. 1. 1.]]
```

Create a random array of 10×10: perform stats (min, max, mean, std)

```
In [45]: arr_random = np.random.random((10,10))
arr_random
```

```
Out[45]: array([[0.56075023, 0.34186493, 0.03468818, 0.45987227, 0.2754969 ,
                0.52016829, 0.5322423 , 0.35348889, 0.82587913, 0.78945678],
               [0.54183899, 0.03062537, 0.72813261, 0.5602377 , 0.42588856,
                0.46824427, 0.38307056, 0.97892666, 0.45878008, 0.90281507],
               [0.48372797, 0.7672162 , 0.84353032, 0.6500642 , 0.10130807,
                0.61508837, 0.97194774, 0.90163551, 0.83278911, 0.95927594],
               [0.10119545, 0.05467544, 0.98564903, 0.23252783, 0.94055784,
                0.96854962, 0.50222156, 0.48577281, 0.87683283, 0.73728299],
               [0.89912782, 0.74047958, 0.84422662, 0.9927838 , 0.86168849,
                0.44965982, 0.72215937, 0.51954719, 0.42604492, 0.62632716],
               [0.90121433, 0.4413712 , 0.8418349 , 0.067059 , 0.85281719,
                0.34725038, 0.87589565, 0.17124562, 0.45270557, 0.65088174],
               [0.39363327, 0.01104553, 0.10551465, 0.41491295, 0.62839635,
                0.36473251, 0.02748165, 0.10134871, 0.89057459, 0.81241118],
               [0.15772141, 0.83460038, 0.89545232, 0.55701763, 0.57305814,
                0.00769877, 0.19545642, 0.97577204, 0.66037023, 0.23837872],
               [0.74654949, 0.56773757, 0.05077119, 0.03825578, 0.46548196,
                0.403564 , 0.41072183, 0.55741628, 0.99760267, 0.52096232],
               [0.07896999, 0.37708798, 0.25504251, 0.22010474, 0.42387439,
                0.98067056, 0.50004804, 0.08702283, 0.75290824, 0.43736246]])
```

```
In [46]: arr_random.min()
```

```
Out[46]: 0.007698765157061804
```

```
In [47]: arr_random.max()
```

```
Out[47]: 0.9976026673241755
```

```
In [48]: arr_random.mean()
```

```
Out[48]: 0.5358636920544533
```

```
In [49]: arr_random.std()
```

```
Out[49]: 0.29664656258817484
```

Broadcast a 1D array over a 2D array [all possible scenarios]

```
In [50]: arr = np.array([1,2,3])
arr
```

```
Out[50]: array([1, 2, 3])
```

```
In [51]: arr2 = np.array([[1,2,3],[4,5,6],[7,8,9]])
arr2
```

```
Out[51]: array([[1, 2, 3],
               [4, 5, 6],
               [7, 8, 9]])
```

```
In [52]: broadcast = arr + arr2
broadcast
```

```
Out[52]: array([[ 2,  4,  6],
               [ 5,  7,  9],
               [ 8, 10, 12]])
```

```
In [53]: broadcast2 = arr - arr2
broadcast2
```

```
Out[53]: array([[ 0,  0,  0],
               [-3, -3, -3],
               [-6, -6, -6]])
```

```
In [54]: broadcast3 = arr * arr2  
broadcast3
```

```
Out[54]: array([[ 1,  4,  9],  
               [ 4, 10, 18],  
               [ 7, 16, 27]])
```

```
In [55]: broadcast4 = arr / arr2  
broadcast4
```

```
Out[55]: array([[1.         , 1.         , 1.         ],  
               [0.25        , 0.4         , 0.5         ],  
               [0.14285714, 0.25        , 0.33333333]])
```

Sort a 2D array by column 1 [hint: check method argsort()]

```
In [56]: arr2
```

```
Out[56]: array([[1, 2, 3],  
               [4, 5, 6],  
               [7, 8, 9]])
```

```
In [57]: sorted = arr2[:,0].argsort()  
sorted
```

```
Out[57]: array([0, 1, 2], dtype=int64)
```

```
In [58]: arr2[sorted]
```

```
Out[58]: array([[1, 2, 3],  
               [4, 5, 6],  
               [7, 8, 9]])
```